

Breaking and Fixing Password-Hardening

Ruben Baecker¹, Paul Gerhart², and Dominique Schröder^{2,1}

¹ Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany
`ruben.baecker@fau.de`

² TU Wien, Vienna, Austria
`{paul.gerhart, dominique.schroeder}@tuwien.ac.at`

Abstract. Password Hardening (PH) is one of the most practical cryptographic solutions for improving authentication security in the presence of low-entropy passwords. It preserves the usability of traditional username–password authentication while adding a cryptographic ratelimiter that prevents offline brute-force attacks even if one party is compromised. However, the existing security model for PH introduced by Lai *et al.* (USENIX’17) assumes static corruptions and fails to capture realistic adversarial behavior. Recent work by Baecker *et al.* (AC’25) on Password-Hardened Encryption (PHE) has shown that such limitations can lead to exploitable vulnerabilities in deployed systems. Motivated by these findings, we revisit password hardening and demonstrate new attacks against both Phoenix (USENIX’17) and PW-Hero (SRDS’22).

To overcome these shortcomings, we introduce a strengthened and unified security definition for PH in the adaptive corruption setting. Building on these foundations, we present *IrmGUARD*, the first PH scheme secure under the new model. Our prototype implementation demonstrates that IrmGUARD exceeds the efficiency of prior schemes, offering the strongest and most deployable password-hardening solution to date.

1 Introduction

Passwords continue to underpin digital authentication despite decades of research into alternatives. Their persistence stems from a pragmatic balance of usability, deployability, and cost: passwords require no special hardware, integrate seamlessly with legacy systems, and remain familiar to users. Yet this convenience comes at a price. As the web consolidates around large identity providers and single sign-on (SSO) ecosystems such as Google and Apple, password storage has become increasingly centralized. A single breach at one of these providers can compromise access to thousands of dependent services, amplifying the impact of each leak. In such settings, the long-standing practice of storing salted hashes, a technique originating in the 1970s, offers little protection once a database is exposed, leaving passwords vulnerable to large-scale offline brute-force attacks. The urgent need to address this problem motivated the invention of password hardening.

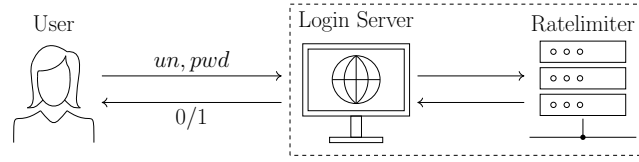


Fig. 1. Intuition of password hardening. The user-facing interface remains unchanged, while password validation requires joint interaction between the server and the ratelimiter.

Password Hardening (PH). Password hardening, originally proposed by Facebook and formalized by Everspaugh *et al.* [13], offers the most practical cryptographic solution to the long-standing password storage problem. It preserves the familiar username–password interface for users while augmenting the server backend with an additional cryptographic service, the ratelimiter. The server and the ratelimiter store the authentication tuple (un, pwd) in a distributed manner, effectively secret-sharing the encrypted password two-out-of-two. This joint storage ensures that neither party can verify passwords independently: as long as one party remains uncompromised, offline brute-force attacks are infeasible. In addition, both parties enforce per-user rate-limits during authentication attempts, mitigating online brute-force attacks. For proactive security or in response to a breach, the server and ratelimiter can execute a lightweight key-rotation protocol that refreshes their cryptographic keys and updates all enrollment records without user involvement. The rotation protocol requires only constant communication overhead, independent of the number of enrolled users. A high-level overview of the user-facing interface in a PH scheme is shown in Figure 1.

Relation to SSO. In recent years, single sign-on (SSO) services such as Apple, Google, and Facebook have become widely adopted. These systems delegate authentication to a central identity provider: when a user attempts to log in to a service, the service forwards the authentication request to the identity provider, which verifies the user’s credentials and returns a signed token confirming successful authentication. While convenient, SSO does not eliminate passwords; instead, it concentrates password storage at a few large providers, creating high-value targets for attackers. This centralization further underscores the need for stronger cryptographic protection of stored passwords. *Eigentlich können wir sagen, dass PH auch bei SSO hilfreich ist oder? –Paul*

Deficiencies of PH. The concept of password hardening was first formalized by Everspaugh *et al.* [13], establishing the foundation for protecting passwords against offline brute-force attacks through cooperation between a server and a ratelimiter. Subsequent work [18,17] refined this idea and inspired the devel-

opment of Password-Hardened Encryption (PHE) [16], later commercialized by Virgil Security,³ and its threshold variant tPHE [8].

However, Beacker *et al.* [5] recently demonstrated that SimplePHE, the PHE scheme underlying the commercial deployment, is vulnerable to a practical password recovery attack stemming from an insufficient security model. The model assumes only static corruption of parties, whereas real deployments require security under alternating corruptions, where the server and ratelimiter may be compromised at different times. Although SimplePHE includes a key-rotation mechanism intended to restore security after compromise, the attack shows that this protection fails under alternating corruption. In addition, Abdolmaleki *et al.* [1] uncovered a proof gap in the tPHE protocol of Brost *et al.* [8].

Together, these findings exposed fundamental weaknesses in the foundations of the password-hardening family. While prior works [5,1] proposed stronger definitions and constructions for PHE, the original password-hardening (PH) setting lacks comparable formal rigor. Since the formal models of PH share the deficiencies of earlier PHE definitions, a natural question arises:

Are existing PH schemes truly secure, or do they suffer from similar flaws?

Why PH still matters. Although PHE offers stronger functionality by combining password hardening with data encryption, pure password hardening remains highly relevant. Many services, including single sign-on providers and web authentication platforms, do not handle encrypted data and thus do not need additional encryption capabilities. In such settings, PH provides a lightweight and efficient alternative that preserves the same resistance to offline and online attacks while avoiding the complexity and computational overhead inherent to PHE. Consequently, PH remains the most practical choice for large-scale authentication infrastructures that prioritize minimal latency and system simplicity.

1.1 Our Contribution

We revisit password hardening and advance its theoretical and practical foundations through the following contributions:

- **Attacks on all existing PH schemes.** We identify and exploit critical vulnerabilities in all existing password-hardening schemes, Phoenix [17] and PW-Hero [15]. The attack on Phoenix leverages an alternating corruption pattern but, unlike prior work [5], remains completely stealthy and undetectable by the server. The attack on PW-Hero succeeds even without alternating corruptions and requires only a malicious server, demonstrating a fundamental design flaw. We implemented both attacks against reference implementations to show the practical relevance.

³ <https://developer.virgilsecurity.com/docs/purekit/fundamentals/password-hardened-encryption/>

- **Refined and unified security model.** We consolidate and strengthen the existing security notions for password-hardening schemes into a single *adaptive* security model. Our security definition, called *Confidentiality*, generalizes prior static notions and aligns the security guarantees of PH with the strong definitions established for Password-Hardened Encryption [5].
- **A new secure and efficient construction.** We present a novel password-hardening scheme, IrmGUARD, prove its security, and benchmark its performance. Our evaluation shows that IrmGUARD is not just the only adaptively secure PH scheme, but also the most efficient one.

1.2 Related Work

Everspaugh *et al.* [13] initiated password-hardening (PH) research by introducing a construction based on partially oblivious pseudorandom functions. Subsequent improvements like PO-COM [18] were later shown to be vulnerable to offline attacks. While Phoenix [17] was proven secure under static corruption, we show in Section 3 that its security fails in a semi-adaptive setting. Similarly, PW-Hero [15] claims superior security to Phoenix but remains vulnerable to a malicious server as demonstrated in Section 3.

Parallel research into Password-Hardened Encryption (PHE) integrated message encryption into the primitive. Although SimplePHE [16] was found insecure in semi-adaptive models, Baecker *et al.* [5] introduced the refined and provably secure HildeGUARD scheme. Brost *et al.* [8] generalized PHE to the threshold setting, but Abdolmaleki *et al.* [1] identified a proof gap in its security analysis and proposed a universally composable construction, based on bilinear pairings.

Other password-based primitives lack essential PH features.

- Password-protected secret sharing (PPSS) [6] and password-authenticated key-exchange (PAKE) [7,14] both require per-user server keys and lack rotation support.
- Distributed Password Verification [9] depends on offline backup tapes for rotation and necessitates separate keys per user.
- Password-based threshold authentication (PbTA) [2] changes the standard user-server interface by requiring verifiable authentication tokens.
- TLS-based password hardening [12] provides an easily deployable alternative but is prone to offline dictionary attacks and lacks rotation capabilities.

2 Preliminaries

This section introduces the notation and assumptions underlying our proofs, recalls the notion of Non-Interactive Zero-Knowledge proofs (NIZKs), and formalizes password hardening.

Notation. Let $\lambda \in \mathbb{N}$ be the security parameter. We refer to the server and the ratelimiter with S and R, respectively. By $x \leftarrow y$, we denote the assignment of y to x , and $x \leftarrow \$ S$ denotes that x is uniformly sampled from set S . We use $y \leftarrow A(x)$ to denote that algorithm A is executed on input x and its output is y . All algorithms are probabilistic polynomial time (PPT) if not stated otherwise. We denote by $(u, v) \leftarrow \langle A(x), B(y) \rangle_P$ the execution of protocol P between the interactive algorithms A and B with their local inputs x and y , and their local outputs u and v . The empty string is denoted by ϵ .

2.1 Hardness Assumptions

Definition 1 (DDH). *The Diffie-Hellman assumption holds in a group $\mathbb{G}$ of prime order q if for any PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that*

$$|\Pr[\text{DDH}_{\mathcal{A}}^0(\mathbb{G}) = 1] - \Pr[\text{DDH}_{\mathcal{A}}^1(\mathbb{G}) = 1]| \leq \text{negl}(\lambda),$$

where the randomness is taken over the random coins of the experiment and the adversary.

$\text{DDH}_{\mathcal{A}}^b(\mathbb{G})$
$x, y, z \leftarrow \$ \mathbb{Z}_q^*$ $X \leftarrow g^x, Y \leftarrow g^y$ if b then $Z \leftarrow g^{xy}$ else $Z \leftarrow g^z$ return $\mathcal{A}(X, Y, Z)$

Fig. 2. Security game for DDH.

Definition 2 (OM-gapDH). *The One-More Gap Diffie-Hellman assumption holds in a group $\mathbb{G}$ of prime order q if for any PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\Pr[\text{OM-gapDH}_{\mathcal{A}}(\mathbb{G}) = 1] \leq \text{negl}(\lambda),$$

where the randomness is taken over the random coins of the experiment and the adversary.

2.2 Non-Interactive Zero-Knowledge Proofs

Let $R : \{0, 1\} \leftarrow \{0, 1\}^* \times \{0, 1\}^*$ be a NP-witness-relation with NP-language $\mathcal{L} := \{x \mid \exists \omega : R(x, \omega) = 1\}$. A non-interactive zero-knowledge proof (NIZK)

OM-gapDH _A (G)	Targ()	Help(z)
$k \leftarrow \mathbb{Z}_q^*$	$c_t \leftarrow c_t + 1$	$c \leftarrow c + 1$
$c, c_t \leftarrow 0$	$x_{c_t} \leftarrow \mathbb{G}$	return z^k
$\mathbb{O} := \{\text{Targ}, \text{Help}, \text{DDH}\}$	return x_{c_t}	
$(i_1, \sigma_1), \dots, (i_\ell, \sigma_\ell) \leftarrow \mathcal{A}^\mathbb{O}(g^k)$	$\text{DDH}(x_1, y_1, x_2, y_2)$	
assert $c < \ell$	$d_1 \leftarrow \text{dlog}_{x_1}(y_1)$	
assert $\forall \alpha : i_\alpha \leq c_t$	$d_2 \leftarrow \text{dlog}_{x_2}(y_2)$	
assert $\forall \alpha \neq \beta : i_\alpha \neq i_\beta$	return $d_1 = d_2$	
return $\forall \alpha : x_{i_\alpha}^k = \sigma_\alpha$		

Fig. 3. Security game for OM-gapDH.

system [11] for the relation R consists of the three efficient algorithms $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, $\pi \leftarrow \text{Prove}(\text{crs}, x, \omega)$, and $b \leftarrow \text{Verify}(\text{crs}, x, \pi)$. The setup algorithm inputs the security parameter and returns a common reference string. The prove algorithm takes as input crs and a statement x with a witness ω and generates a proof π . The verify algorithm takes as input crs , x , and π and returns $b = 1$ if the proof is valid and $b = 0$ otherwise. We use a NIZK system that is: (1) complete, requiring that for every $x \in L$, Prove generates proofs that Verify accepts,

$$\begin{aligned} \Pr[\text{Verify}(\text{crs}, x, \pi) = 1 \mid \text{crs} \leftarrow \text{Setup}(1^\lambda), \pi \leftarrow \text{Prove}(\text{crs}, x, \omega)] \\ \geq 1 - \text{negl}(\lambda) \quad \forall x \in L, \end{aligned}$$

(2) sound, meaning that for every $x \notin L$, it is hard for an adversary $\mathcal{A}$ to generate proofs that Verify accepts,

$$\Pr[\text{Verify}(\text{crs}, x, \pi) = 1 \mid \text{crs} \leftarrow \text{Setup}(1^\lambda), \pi \leftarrow \mathcal{A}(\text{crs}, x, \omega)] \leq \text{negl}(\lambda) \quad \forall x \notin L,$$

and (3) zero-knowledge, ensuring that the verifier learns nothing beyond the validity of the statement x . This property is proven by constructing a simulator $\mathcal{S}$ that computes proofs without knowing the witness (but with some trapdoors) that are indistinguishable from “real” proofs.

$$\left| \Pr[\mathcal{A}^{\mathcal{S}(\text{crs}, x)} = 1] - \Pr[\mathcal{A}^{\text{Verify}(\text{crs}, x, \omega)} = 1] \right| \leq \text{negl}(\lambda).$$

2.3 Password Hardening

Definition 3 (Password Hardening [17]). A password hardening (PH) scheme, consists of four efficient algorithms (Setup , SKeyGen , RKeyGen , Update) and three efficient protocols $(\langle \text{S}, \text{R} \rangle_{\text{enr1}}, \langle \text{S}, \text{R} \rangle_{\text{val}}, \langle \text{S}, \text{R} \rangle_{\text{rot}})$, that we define as follows:
 $pp \leftarrow \text{Setup}(1^\lambda)$: The setup algorithm takes the security parameter 1^λ as input and returns public parameters pp .

$(pk_S, sk_S) \leftarrow \text{\$SKeyGen}(pp)$: The server key generation algorithm takes the public parameter pp as input and returns a server key pair (pk_S, sk_S) .

$(pk_R, sk_R) \leftarrow \text{\$RKeyGen}(pp)$: The ratelimiter key generation algorithm takes the public parameter pp as input and returns a ratelimiter key pair (pk_R, sk_R) .

We assume that all following algorithms take the public keys pk_S, pk_R as an implicit input.

$(T, \epsilon) \leftarrow \text{\$}\langle S(sk_S, un, pwd), R(sk_R, un) \rangle_{\text{enr1}}$: The enrollment protocol is run between the server and the ratelimiter. The server takes its secret key sk_S , the username un , and the password pwd as inputs, while the ratelimiter takes its secret key sk_R and the username un as inputs. The server returns an enrollment record T .

$(b, \epsilon) \leftarrow \text{\$}\langle S(sk_S, T, un, pwd'), R(sk_R, un) \rangle_{\text{val}}$: The validation protocol is run between the server and the ratelimiter. The server takes its secret key sk_S , the enrollment record T , the username un , and the candidate password pwd' as inputs, while the ratelimiter takes its secret key sk_R and the username un as inputs. The server returns a decision bit b that indicates whether the authentication attempt was successful.

$((sk'_S, pk'_R, \tau), (sk'_R, pk'_S)) \leftarrow \text{\$}\langle S(sk_S), R(sk_R) \rangle_{\text{rot}}$: The key rotation protocol is run between the server and the ratelimiter. The server and the ratelimiter take their secret key sk_S and sk_R , respectively, as inputs. The server returns its new secret key sk'_S , the ratelimiter's updated public key pk'_R , and an update token τ . The ratelimiter returns its new secret key sk'_R and the server's updated public key pk'_S .

$T' \leftarrow \text{Update}(sk_S, \tau, T, un)$: The update algorithm takes the server's secret key sk_S , the update token τ , the enrollment record T , and the username un as inputs.

The existing security properties of password hardening schemes comprise four notions under a *static* corruption model [17], i.e., the adversary corrupts either the server or the ratelimiter, but not both over time. In particular, no guarantees hold under sequential corruptions (e.g., server compromise followed by ratelimiter compromise after key rotation). We briefly summarize the properties below and refer to [17] for formal definitions.

- **Partial Obliviousness.** Even if the server enrolls one of two adversarially chosen passwords, a malicious ratelimiter cannot distinguish which password was used.
- **Hiding.** Given sk_S and all enrollment records, a malicious server gains no advantage beyond online brute-force via interaction with the ratelimiter.
- **Forward Security.** After key rotation, updated enrollment records are indistinguishable from fresh ones under new keys.
- **Binding.** A malicious ratelimiter cannot make an honest server accept two distinct username–password pairs for the same enrollment record.

3 Breaking Password Hardening

In this section, we present attacks on all the existing PH schemes Phoenix and PW-Hero. For clarity, we strip each protocol down to the components that are relevant to the attacks; the omitted details do not affect the attacks' feasibility.

Breaking Phoenix. In Phoenix, a user's password is represented as a PRF output under the server's secret key and then blinded with a second PRF output computed from a user-specific nonce under the ratelimiter's secret key. The resulting dual-PRF value is encrypted under the ratelimiter's ElGamal public key, preventing the server from directly reading the record components.

Phoenix utilizes two hash functions H_R and H_S . The server's secret key is denoted by sk_S , and the per-user randomness is denoted by r . We assume the target user is already enrolled with the secret password secPW . The honest enrollment record for this user is

$$T^{\text{hon}} = (t_1^{\text{hon}}, t_2^{\text{hon}}, n_S^{\text{hon}}, n_R^{\text{hon}})$$

with

$$t_1^{\text{hon}} = g^r, \quad t_2^{\text{hon}} = h^r \cdot H_S(un^{\text{hon}}, \text{secPW}, n_S^{\text{hon}})^{sk_S} \cdot H_R(un^{\text{hon}}, n_R^{\text{hon}})^{sk_R},$$

where s is the ratelimiter's ElGamal secret key and $h = g^s$ its public key. As usual for PH/PHE, we assume full adversarial read access to the server database.

Step 1 - Ratelimiter Corruption: The adversary first corrupts the ratelimiter and learns s and sk_R . Using s , it can decrypt t_2^{hon} and remove the ratelimiter PRF factor to extract the server PRF term:

$$H_S(un^{\text{hon}}, \text{secPW}, n_S^{\text{hon}})^{sk_S} \leftarrow t_2^{\text{hon}} \cdot (t_1^{\text{hon}})^{-s} \cdot H_R(un^{\text{hon}}, n_R^{\text{hon}})^{-sk_R}.$$

Next, the adversary enrolls un^{mal} with password malPW , and injects the extracted server PRF into the ratelimiter's response by multiplying it to the honest response. Since in Phoenix the server does not verify responses during enrollment, the malicious record's second component becomes malformed:

$$h^{r'} \cdot H_S(un^{\text{mal}}, \text{malPW}, n_S^{\text{mal}})^{sk_S} \cdot H_R(un^{\text{mal}}, n_R^{\text{mal}})^{sk_R} \cdot H_S(un^{\text{hon}}, \text{secPW}, n_S^{\text{hon}})^{sk_S}.$$

The adversary releases the ratelimiter⁴, which becomes honest after the next key rotation. The key rotation updates the ratelimiter's secret keys to s' and sk_R' , and its public key to $\bar{h} = g^{s'}$. The server's secret key is updated to sk_S' .

Step 2 - Server Corruption: After the key rotation, the adversary corrupts the server and learns sk_S' . Because it also knows malPW , it can remove the honest server PRF term for un^{mal} from the updated t_2^{mal} , obtaining

$$\bar{h}^{r'} \cdot H_S(un^{\text{hon}}, \text{secPW}, n_S^{\text{hon}})^{sk_S'} \cdot H_R(un^{\text{mal}}, n_R^{\text{mal}})^{sk_R'}.$$

⁴ As described in [5], this is not an irrational tactic of the adversary but the result of incident response and access revocation.

The final attack step relies on the validation protocol, in which the server divides the server PRF, computed using the candidate password, from t_2 . If the password matches the enrolled password, the server’s PRF factors cancel out, leaving an encryption of the ratelimiter PRF value. The ratelimiter checks the ciphertext and returns a proof of well-formedness.

The corrupt server validates password guesses for un^{hon} by consuming the ratelimiter quota of un^{mal} . It divides the server PRF for a guess with un^{hon} and n_S^{hon} from the modified t_2^{mal} , and sends the resulting ciphertext to the ratelimiter as an authentication attempt for un^{mal} . Because the ratelimiter perceives a correct guess as a successful authentication by un^{mal} , the adversary can validate guesses for un^{hon} until the rate limit for un^{mal} is exhausted. The adversary aggregates the quota of multiple malicious accounts to accelerate an online brute-force attack against **secPW**.

In summary, the attack arises from two facts: (i) a corrupted ratelimiter can decrypt an honest enrollment record and extract the server PRF term (i.e., a value derived from the honest user’s password), and (ii) the ratelimiter’s enrollment response is not checked for well-formedness.

We confirmed the feasibility of this attack against a reference implementation. Our implementation successfully bypasses the rate-limiting protections, which underscores the danger of relying on weaker security models and highlights the gap in the protocol logic. We include our implementation in our artifact.

Breaking PW-Hero. Since PW-Hero [15] is the only remaining unbroken PH candidate, we investigated whether the vulnerabilities identified in Phoenix also apply to this construction. Our analysis reveals an even more severe flaw in the PW-Hero protocol that persists even without an alternating corruption pattern. In other words, PW-Hero is broken. [Changed paragraph–Paul](#)

On an intuitive level, the server and the ratelimiter in PW-Hero run a sub-protocol to compute a pseudorandom value that depends on the username un , the password pwd , and the ratelimiter’s secret key sk_R . The resulting value y is then stored in the enrollment record. During validation, the sub-protocol is re-run with candidate password pwd' to obtain y' , which confirms the password is correct if it matches the stored value y .

*The setting*⁵: We assume that the target user is already enrolled with the secret password **secPW**. The honest enrollment record for this user is

$$T = (t_1, t_2, n)$$

with

$$t_1 = \rho \cdot H(un, pwd, n), \quad t_2 = \left(\frac{t_1}{H(un, pwd, n)} \cdot un \right)^{sk_R} = (\rho \cdot un)^{sk_R},$$

⁵ We describe a simplified version of PW-Hero here. These simplifications do not affect the feasibility of the attack.

where ρ is a user-specific, uniformly random value.

Honest Validation: We recall the validation protocol. The server computes the salted hash of the password and deblinds t_1 to recover $\rho' \leftarrow t_1 / \mathbf{H}(un, pwd', n)$. It sends blinded values $x_1 \leftarrow \rho'^r$ and $x_2 \leftarrow un^r$, and the unblinded un to the ratelimiter, along with a proof of well-formedness of x_2 . The ratelimiter uses un to enforce per-user rate-limits. The ratelimiter validates the proof, multiplies x_1 with x_2 and raises it to its secret key, yielding $y \leftarrow (x_1 \cdot x_2)^{sk_s} = (\rho \cdot un)^{r \cdot sk_s}$. The ratelimiter sends y to the server, which checks whether it matches t_2 .

Attack as a malicious Server: A malicious server can bypass rate-limiting for user un because the well-formedness proof only covers x_2 . By sending

$$x_1^* \leftarrow x_1 \cdot \frac{un^r}{un'^r}$$

along with un' and $x_2' = un'^r$, the server induces the ratelimiter to use the quota of un' . The modification of x_1^* cancels out the multiplication with $x_2' = un'^r$ resulting in

$$\tilde{y} \leftarrow (x_1^* \cdot x_2')^{sk_R} = \left(x_1 \cdot \frac{un^r}{un'^r} \cdot un'^r \right)^{sk_R} = (x_1 \cdot un^r)^{sk_R}$$

which is exactly what we would expect for a computation involving user un . To execute an online dictionary attack against un , the adversary repeats this procedure for each guess by selecting an arbitrary user un' with available quota to aggregate the rate limits of multiple accounts until the correct password is recovered.

We also provide a reference implementation of PW-Hero and demonstrate the feasibility of our attack in our artifact.

4 Enhanced Security Definitions for PH

The attacks in the previous section reveal that current static corruption models for password hardening fail to capture realistic adversarial behavior. To address these weaknesses, we propose an enhanced security definition for PH that extends the framework to a semi-adaptive corruption model.

Confidentiality Current PH security definitions treat attack vectors as independent games, which fails to capture adversaries who combine multiple vectors. Because these models are restricted to static corruption, an adversary can never corrupt both parties, neither simultaneously nor sequentially. Our attacks on Phoenix and PW-Hero prove that this fragmented approach results in practical vulnerabilities that lead to password recovery and rate-limiting bypasses. To resolve this, we introduce *Confidentiality*, a unified definition that covers partial obliviousness, hiding, forward security, and binding under the adaptive corruption model for both parties.

Baecker *et al.* unified security definitions for the related notion of password-hardened encryption in a similar effort [5]. Unfortunately, formalizing the security of PH and PHE differs significantly in their nature despite their supposed similarity. For PHE, the adversary must not learn the uniformly random key encrypted in a ciphertext, unless it guesses the password within a certain number of interactions with the honest parties. For PH, we want to ensure that the adversary cannot successfully authenticate to the honest server as someone else. The difference is reflected in the type of winning condition, with *learning* something for PHE and *convincing* someone for PH. We formalize the winning condition by introducing a solve oracle that allows the adversary to interact with the honest server in a validation protocol for a challenge user un^* .

Corruption Model. Security is defined in epochs where honest key rotations reset any existing corruption. Per epoch, either the server or the ratelimiter may be corrupted because protecting against simultaneous corruption is generally impossible. While prior work [8,5,1] focused on semi-adaptive corruption where the adversary announces its target before the next rotation, we target full adaptive security where the adversary can corrupt parties at any time during the protocol execution. While a corrupted server naturally learns passwords from active enrollment or validation requests, all other passwords must remain secure against brute-force attacks. A corrupted ratelimiter may disrupt service by refusing to respond, but must not learn any passwords or independently brute-force leaked records. It also cannot convince an honest server to accept authentication attempts using incorrect passwords.

Enforced Rate-limiting. Existing PH definitions artificially limit validation queries in security games, which permits schemes to be proven secure even without implementing rate-limiting. Furthermore, current schemes rely solely on the ratelimiter for rate-limiting, allowing a compromised ratelimiter to bypass quota through unrestricted server interactions. We address this by allowing unlimited validation oracle queries in the security games to force the protocol to implement rate-limiting on both the server and the ratelimiter. This approach follows Baecker *et al.* [5], who addressed this issue in PHE.

The Confidentiality Game. The security game samples keys for the server and ratelimiter, samples a challenge username un^* from a fixed username space $\mathcal{UN}$, and a challenge password pwd^* from a fixed password space $\mathcal{PW}$. The challenge password pwd^* is used in an enrollment protocol to generate an enrollment record T^* for the challenge username un^* . The adversary wins the game if it successfully authenticates to an honest server as un^* . Furthermore, the adversary can interact with honest parties through multiple oracles, which we intuitively describe in the following. We formalize the oracles in the appendix in Figure 5. Enrollment & Validation Oracle: The oracle runs the corresponding protocol with the adversary, simulating the non-corrupted party. It returns the enrollment record and the validation result b , respectively. The enrollment oracle can-

$\text{Confidentiality}_{PH, \mathcal{A}, Q_{\text{val}}, \mathcal{UN}, \mathcal{PW}}(\lambda)$	
1 :	$pp \leftarrow \$ \text{Setup}(1^\lambda), (pk_S, sk_S) \leftarrow \$ \text{SKeyGen}(pp), (pk_R, sk_R) \leftarrow \$ \text{RKeyGen}(pp)$
2 :	$\text{Corr} := \perp, un^* \leftarrow \$ \mathcal{UN}, pwd^* \leftarrow \$ \mathcal{PW}, i = 0$
3 :	$\mathbb{O} := \{\mathcal{O}_{\text{enr1}}, \mathcal{O}_{\text{val}}, \mathcal{O}_{\text{enr1Ch}}, \mathcal{O}_{\text{val1Ch}}, \mathcal{O}_{\text{HonRot}}, \mathcal{O}_{\text{DishonRot}}, \mathcal{O}_{\text{Update}}, \mathcal{O}_{\text{Solve}}\}$
4 :	$\mathcal{A}^{\mathbb{O}}(1^\lambda, pk_S, pk_R, un^*)$
5 :	return 0

Fig. 4. The security game of confidentiality for PH.

not be called using the challenge username un^* , as otherwise, a successful attack is trivial by re-enrolling un^* with a known password.

Challenge Oracles: The adversary interacts in the enrollment/validation protocol with the honest server that uses the challenge username and the challenge password. The challenge enrollment oracle can only be called once.

Rotation Oracle: The rotation oracle uses a bit Hon to select between an honest rotation that resets the corruption state or a dishonest rotation. The oracle executes the protocol, potentially interactively with the adversary.

Update Oracle: The update oracle takes an epoch j and uses the corresponding update token τ_j to update an enrollment record T into an enrollment record T' and returns it.

Corruption Oracle: The corruption oracle ensures that no party is corrupt yet, updates the corruption status, and returns the corresponding secret key.

Solve Oracle: The adversary interacts in the validation protocol with the honest server that uses the challenge username. If authentication succeeds, the security game ends and returns 1. This captures exactly what we would expect from a PH scheme: if at least one party is honest, an adversary cannot authenticate as someone else. Because learning the challenge password allows the adversary to authenticate as un^* , confidentiality covers password privacy.

Definition 4 (Confidentiality). *A password hardening scheme PH is adaptively confidential if, for any PPT adversary $\mathcal{A}$, $Q_{\text{val}} \geq 0$, and any password space $\mathcal{PW}$ ⁶, there exists a negligible function negl , such that for all $\lambda \in \mathbb{N}$*

$$\Pr[\text{Confidentiality}_{PH, \mathcal{A}, Q_{\text{val}}, \mathcal{PW}}(\lambda) = 1] \leq \frac{Q_{\text{val}}}{|\mathcal{PW}|} + \text{negl}(\lambda),$$

where Q_{val} is the rate-limiting threshold of the PH scheme. The randomness is taken over the random coins of all randomized algorithms.

⁶ For simplicity, we assume a password space where passwords are equally distributed instead of an arbitrary password distribution. Still, our Confidentiality definition can naturally be generalized to allow for arbitrary password distributions.

$\mathcal{O}_{\text{enr1}}(un, pwd)$	$\mathcal{O}_{\text{val}}(T, un, pwd)$
1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 2 : $\text{else } S(sk_S, un, pwd)$ 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 4 : $\text{else } R(sk_R, un)$ 5 : $(T, \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{enr1}}$ 6 : return T	1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 2 : $\text{else } S(sk_S, T, un, pwd)$ 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 4 : $\text{else } R(sk_R, un)$ 5 : $(b, \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{val}}$ 6 : return b
$\mathcal{O}_{\text{enr1Ch}}()$	$\mathcal{O}_{\text{valCh}}(T)$
1 : ensure $\text{Corr} \neq S \wedge T_j^* = \epsilon \forall j$ 2 : $T_i^* \leftarrow \mathcal{O}_{\text{enr1}}(un^*, pwd^*)$ 3 : return T_i^*	1 : ensure $\text{Corr} \neq S$ 2 : $\epsilon \leftarrow \mathcal{O}_{\text{val}}(T, un^*, pwd^*)$ 3 : return ϵ
$\mathcal{O}_{\text{Rot}}(Hon)$	$\mathcal{O}_{\text{Solve}}(pwd)$
1 : if $Hon = \top$ then $\text{Corr} \leftarrow \emptyset$ 2 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 3 : $\text{else } S(sk_S)$ 4 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 5 : $\text{else } R(sk_R)$ 6 : $((k_S, \tau'), k_R) \leftarrow \langle S^*, R^* \rangle_{\text{rot}}$ 7 : $(pk_R, sk_R) \leftarrow k_R, i \leftarrow i + 1$ 8 : $(pk_S, sk_S) \leftarrow k_S, \tau_i \leftarrow \tau'$ 9 : if $\text{Corr} = S$ then return sk_S 10 : if $\text{Corr} = R$ then return sk_R 11 : return ϵ	1 : ensure $\text{Corr} \neq S \wedge T_i^* \neq \epsilon$ 2 : if $\mathcal{O}_{\text{val}}(un^*, pwd, T_i^*)$ then 3 : end game with 1 4 : else end game with 0
$\mathcal{O}_{\text{Update}}(j, T, un)$	
	1 : ensure $\tau_j \neq \epsilon$ 2 : $T' \leftarrow \text{Update}(sk_{S_j}, \tau_j, T, un)$ 3 : if $T = T_{j-1}^* \wedge un = un^*$ then 4 : $T_j^* \leftarrow T'$ 5 : return T'
$\mathcal{O}_{\text{Corrupt}}(\text{Corr}')$	
1 : ensure $\text{Corr} \notin \{S, R\}$ 2 : $\text{Corr} \leftarrow \text{Corr}'$ 3 : if $\text{Corr} = S$ then return sk_S 4 : if $\text{Corr} = R$ then return sk_R	

Fig. 5. Oracles for the adaptive PH confidentiality definition.

5 Fixing Password Hardening

In this section, we propose **IrmGUARD**⁷, the first password hardening scheme secure in the adaptive corruption model. In the following, let $\mathbb{G}$ be a multiplicative finite cyclic group of order $q = q(\lambda)$, and $H_{\{S, R, 1\}} : \{0, 1\}^* \rightarrow \mathbb{G}$ and $H_{\{N, 2\}} :$

⁷ We name our protocol **IrmGUARD** as a pendant to **HildeGUARD** [5]. The name derives from the German given name "Irmgard" composed of "Irmin" ("universal" or "whole") and "gard" (Germanic for "protection"), reflecting the protocol's role in safeguarding and protecting passwords against dictionary attacks.

Setup(1^λ)	RKeyGen(pp)	SKeyGen(pp)
$crs \leftarrow \Pi_{\text{NIZK}}.\text{Gen}(1^\lambda); g \leftarrow \mathbb{G}$	$sk_R \leftarrow \mathbb{Z}_q^*$	$sk_S \leftarrow \Pi_{\text{Enc}}.\text{Gen}(1^\lambda)$
$H_R, H_S, H_1 \leftarrow \mathcal{H} = \{H : \{0, 1\}^* \rightarrow \mathbb{G}\}$	$pk_R \leftarrow g^{sk_R}$	$pk_S \leftarrow \epsilon$
$H_N, H_2 \leftarrow \mathcal{H} = \{H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}$	return (pk_R, sk_R)	return (pk_S, sk_S)
return $pp \leftarrow (crs, g, H_R, H_S, H_1, H_N, H_2)$		

Fig. 6. The setup algorithms of lrmGUARD.

$\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ are hash functions. Let Π_{NIZK} be a non-interactive zero-knowledge proof system for discrete logarithm representation. We use a standard Chaum-Pedersen style proof [10] for the relation $h_R = H_R(un, n)^{sk_R} \wedge pk_R = g^{sk_R}$. Let Π_{Enc} be a private key encryption scheme. If an assertion fails during the execution of an algorithm, it terminates and returns the special symbol $\perp$ to indicate an error.

In lrmGUARD, a password is stored in a tag t jointly computed by the server and the ratelimiter. The tag consists of two components $t \leftarrow h_S^{-1} \cdot h_R$ where each component is computed by one of the parties. The server's h_S is a standard salted hash of the password $H_S(un, pwd, n)$. The ratelimiter's h_R is a PRF evaluated on the username and the nonce $H_R(un, n)^{sk_R}$, resulting in a value that is easily re-computable by the ratelimiter but looks uniformly random to anyone else, including the server.

To check the validity of an entered password pwd' , the server re-computes h'_S using pwd' and multiplies it by the tag, resulting in h'_R . If the password matches the one stored in t , h'_R matches h_R . To conclude the authentication, the server and the ratelimiter obviously check if $h'_R = h_R$. This oblivious check is done by a protocol that can be seen as an OPRF-based private set intersection protocol where the server's set consists of h'_R and the ratelimiter's set consists of the re-computed h_R . An oblivious pseudorandom function (OPRF) is a secure two-party protocol to evaluate a PRF, where one party (the ratelimiter) holds the key and learns nothing about the input and the output of the computation, and the other party (the server) holds the input to the PRF and learns the output.

Setup Algorithm. The setup algorithm samples a common reference string crs of the proof system Π_{NIZK} , a random generator g of group $\mathbb{G}$, three hash functions H_R, H_S , and H_1 that map arbitrary bit strings to group elements, and two hash functions H_N and H_2 that map arbitrary bit strings to bit strings of length λ . The ratelimiter key generation algorithm samples a secret key sk_R from $\mathbb{Z}_q^*$ and computes the corresponding public key $pk_R \leftarrow g^{sk_R}$. The server key generation algorithm samples a symmetric secret key sk_S . The setup and key-generation algorithms can be seen in Figure 6.

Enrollment Protocol. To enroll a new user un with the password pwd , the server and the ratelimiter both sample a random nonce n_S and n_R , respectively, which are combined to a shared nonce $n \leftarrow H_N(n_S, n_R)$. The ratelimiter computes the

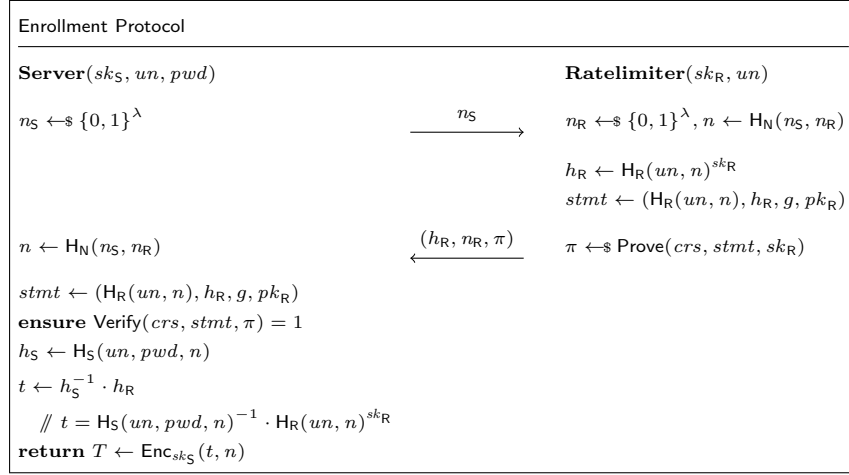
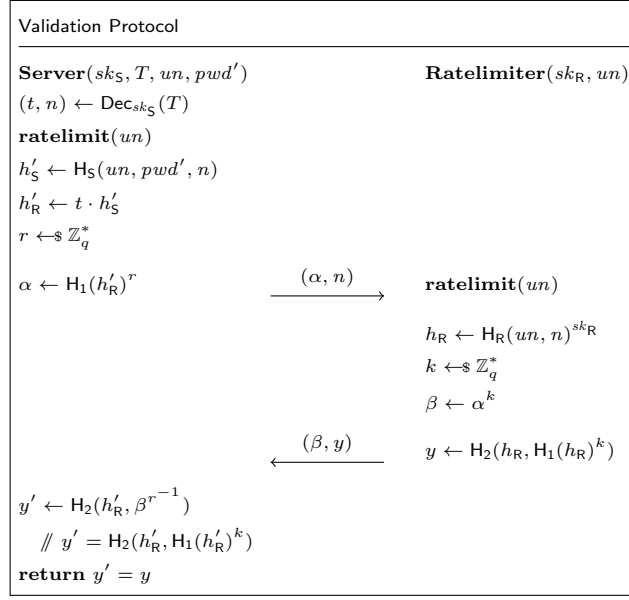


Fig. 7. The enrollment protocol of lrmGUARD.

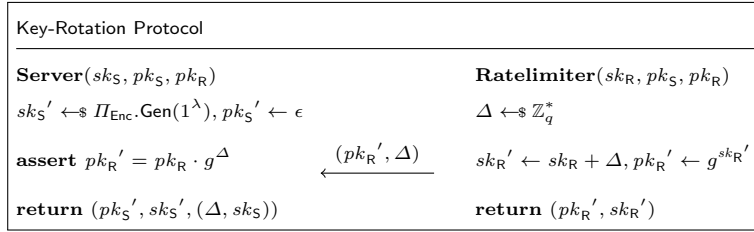
$h_R \leftarrow H_R(un, n)^{sk_R}$ and sends h_R , and a proof of well-formedness to the server. The server computes the hash $h_S \leftarrow H_S(un, pwd, n)$, which binds the username un and the password pwd . Upon receiving h_R , the server verifies the proof of well-formedness and computes $t \leftarrow h_S^{-1} \cdot h_R$, which combines the pseudorandom value of the ratelimiter with the commitment to the username and the password. Afterward, the server encrypts the record t with the nonce n using its secret key sk_S and stores the ciphertext as enrollment record T . We formally describe the enrollment protocol in Figure 7.

Validation Protocol. The validation protocol verifies a candidate password pwd' . To prevent brute-force attacks, a **ratelimit** function counts the requests per user and aborts if the maximum quota Q_{val} is exceeded. The server decrypts the ciphertext to obtain tag t and the nonce n and multiplies t by the hash of the password $H_S(un, pwd', n)$, resulting in $H_R(un, n)^{sk_R}$ if $pwd' = pwd$. The server hashes h'_R and raises it to a uniformly random scalar r , resulting in $\alpha \leftarrow H_1(h'_R)^r$. It sends α and n to the ratelimiter. The ratelimiter re-computes $h_R \leftarrow H_R(un, n)^{sk_R}$, samples an ephemeral key k , and raises α to k , resulting in $\beta \leftarrow \alpha^k$. Then, the ratelimiter hashes h_R , raises it to k , and hashes it again with h_R , resulting in $y \leftarrow H_2(h_R, H_1(h_R)^k)$. The ratelimiter sends β and y to the server, which deblinds β and hashes it with h'_R , resulting in $y' \leftarrow H_2(h'_R, \beta^{r^{-1}})$. If $y' = y$, then the entered password matches the one enrolled in T and the server returns 1 to indicate a successful authentication. Figure 8 shows the validation protocol of lrmGUARD.

Key-Rotation Protocol. The ratelimiter samples a random additive shift Δ , which is added to its current secret key sk_R . The server samples a new, independent encryption key sk'_S . The update token τ consists of the old encryption

**Fig. 8.** The validation protocol of IrmGUARD.

key sk_S and the additive shift Δ . We assume that the server's and ratelimiter's **ratelimit** functions are implicitly synched such that both use the previous epoch's smaller limit in the next epoch. Figure 9 shows the key rotation protocol.

**Fig. 9.** The key rotation protocol of IrmGUARD.

Update Algorithm. To update an entry T , the server decrypts the ciphertext using the old encryption key. Then, the server applies the random shift Δ to t by computing $t' = t \cdot H_R(un, n)^\Delta = h_S^{-1} \cdot H_R(un, n)^{sk_R + \Delta}$. The tag t' is a valid tag for the updated ratelimiter key $sk_R + \Delta$. After updating t , the server encrypts t' with its new encryption key sk' . Figure 10 shows the update algorithm.

$\text{Update}(sk_S, \tau, T, un)$
parse τ as $(\Delta, s\bar{k}_S)$ $(t, n) \leftarrow \text{Dec}_{s\bar{k}_S}(T)$ $T' \leftarrow \text{Enc}_{sk_S}(t \cdot \text{H}_R(un, n)^\Delta, n)$ return T'

Fig. 10. The update algorithm of IrmGUARD.

5.1 Security of IrmGUARD

Theorem 1. *Let Π_{Enc} be a CCA-secure private key encryption scheme, Π_{NIZK} be a complete, sound, and zero-knowledge NIZK proof system, and OM-gapDH and DDH be hard in $\mathbb{G}$. If the hash functions H_S , H_R , H_N , H_1 , and H_2 are modeled as random oracles, then IrmGUARD is confidential.*

We prove the semi-adaptive security of the scheme first and use a compiler, described in Section 5.1, to lift the semi-adaptive security to full adaptive security. The oracles for the semi-adaptive security experiment are formally defined in Figure 12.

Proof (Semi-Adaptive Confidentiality). The idea of our proof is to incrementally update the challenge ciphertext, through various proof hops, so that it eventually consists only of random elements. Once the ciphertext is independent of the password, the only way to obtain information is by querying the oracles, and the adversary's probability of success reduces to the guessing probability. We prove confidentiality through a series of game hops.

$\text{Exp}_0(\lambda)$. The initial experiment is identical to the confidentiality experiment. Hence, it holds that

$$\Pr[\text{Confidentiality}_{PH, \mathcal{A}, Q_{\text{val}}, \mathcal{UN}, \mathcal{PW}}(\lambda) = 1] = \Pr[\text{Exp}_0(\lambda) = 1].$$

$\text{Exp}_1(\lambda)$. This experiment differs from the previous only in the way we simulate the random oracle H_R . We respond to unseen queries with g^a for a uniformly sampled exponent $a \leftarrow \mathbb{Z}_q^*$. We keep track of the inputs and the corresponding exponents a , and respond to already-seen inputs with the same g^a to ensure consistency throughout different queries. This experiment is functionally equivalent to the previous one and therefore

$$\Pr[\text{Exp}_0(\lambda) = 1] = \Pr[\text{Exp}_1(\lambda) = 1].$$

$\text{Exp}_2(\lambda)$. This experiment differs from the previous only in the way we handle collisions within the random oracles H_S , H_R , H_2 , and H_N : if a collision occurs, we abort the experiment. The probability of a collision occurring in each random oracle can be upper bounded by the birthday paradox, resulting in $\Pr[\text{coll}_{\text{H}_x}] \leq \frac{q\text{H}_x}{|R|}$

with q_{H_x} being the number of queries to H_x and R being the range of the random oracle. While all q_{H_x} are at most polynomial in the security parameter (because the adversary is PPT), the domains of all random oracles are of exponential size in the security parameter, resulting in a negligible probability for an abort. Exp_2 is only distinguishable from Exp_1 , if it aborts and therefore

$$|\Pr[\text{Exp}_1(\lambda) = 1] - \Pr[\text{Exp}_2(\lambda) = 1]| \leq \text{negl}(\lambda).$$

$\text{Exp}_3(\lambda)$. This experiment is functionally equivalent to $\text{Exp}_2(\lambda)$ except that we abort if the first invocation of the random oracle H_R on the challenge nonce n^* is not during the challenge enrollment. Since $n_\mathcal{S}^*$ is sampled uniformly at random and H_N has no collisions, this only happens if, by chance, $n_\mathcal{S}^*$ is sampled twice, or n^* is equivalent to any input given in previous oracle calls. The probability of this happening is $\frac{q_{\text{enroll}}}{2^\lambda} + \frac{q_{H_R}}{2^\lambda}$ with q_{enroll} being the number of queries to $\mathcal{O}_{\text{enr1}}$ and q_{H_R} being the number of queries to H_R . Because the adversary is PPT, q_{enroll} and q_{H_R} are polynomially bound, resulting in a negligible probability for an abort. Exp_3 is only distinguishable from Exp_2 , if it aborts and therefore

$$|\Pr[\text{Exp}_2(\lambda) = 1] - \Pr[\text{Exp}_3(\lambda) = 1]| \leq \text{negl}(\lambda).$$

$\text{Exp}_4(\lambda)$. In this experiment, we make use of the OM-gapDH and DDH assumption defined in Definitions 1 and 2. During the challenge enrollment, we defer from our usual way of answering queries to H_R on input (un^*, n^*) . Instead of sampling a fresh exponent a , we inject the DDH challenge (X, Y, Z) by responding with Y to (un^*, n^*) . We store Y with (un^*, n^*) but without an exponent a' such that $Y = g^{a'}$.

Furthermore, we answer all queries to H_1 with challenge elements from Targ . This experiment is functionally equivalent to the previous one and therefore

$$\Pr[\text{Exp}_3(\lambda) = 1] = \Pr[\text{Exp}_4(\lambda) = 1].$$

We now build hybrids over each invocation of an honest key rotation.

$\text{Exp}_{4,0}(\lambda)$. This experiment is equivalent to $\text{Exp}_4(\lambda)$.

$\text{Exp}_{4,q_{\text{rot}}}(\lambda)$. In this experiment, the challenge enrollment record T^* is independent from the challenge password pwd^* during all epochs.

$\text{Exp}_{4,i}(\lambda)$. This experiment differs from the previous one ($\text{Exp}_{4,i-1}(\lambda)$) in the epoche after the i -th key-rotation. We distinguish between two cases depending on the corruption during this epoch. We assume, without loss of generality, that the adversary corrupts either the server or the ratelimiter.

Case 1: Corruption of S .

$\text{Exp}_{4,i,0}^S(\lambda)$. This experiment is equivalent to $\text{Exp}_{4,i}(\lambda)$.

$\text{Exp}_{4,i,1}^S(\lambda)$. In this experiment, we compute the NIZK proofs sent by the ratelimiter during enrollment using a simulator that can generate proofs for a statement without knowing the corresponding witness. Such a simulator is guaranteed to exist due to the zero-knowledge property of the proof system Π_{NIZK} . Because $\text{Exp}_{4,i,1}^S(\lambda)$ only differs from $\text{Exp}_{4,i,0}^S(\lambda)$ in the simulated proofs, every adversary that distinguishes $\text{Exp}_{4,i,1}^S(\lambda)$ from $\text{Exp}_{4,i,0}^S(\lambda)$ also distinguishes the simulated proofs from the real proofs. Therefore, the adversary also breaks the zero-knowledge property of the proof system. Consequently, it holds that

$$\left| \Pr[\text{Exp}_{4,i,0}^S(\lambda) = 1] - \Pr[\text{Exp}_{4,i,1}^S(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

$\text{Exp}_{4,i,2}^S(\lambda)$. In this experiment, we replace the hashes of the challenge nonce raised to the ratelimiter secret key $H_R(un^*, n^*)^{sk_R}$ with a uniformly random value Z . Instead of sampling a fresh ratelimiter key, we use the first component of the DDH-tuple (X, Y, Z) from Exp_4 as the ratelimiter's public key $pk_R = X$. Note that this is indistinguishable in the adversary's view because it never sees τ and there is always a matching Δ for every pair of old and new ratelimiter key $\Delta \leftarrow sk_R' - sk_R$. During enrollment and validation protocols, we compute $H_R(un, n)^{sk_R} = (g^a)^{sk_R}$ as X^a , with a stored in Exp_1 . For the challenge nonce n^* , we compute $H_R(un^*, n^*)^{sk_R} = Y^{sk_R}$ as Z . When the challenge enrollment record is updated during the update protocol, we extract $H_R(un^*, n^*)^{sk_R}$ from t and replace it with Z . If there exists an adversary that distinguishes $\text{Exp}_{4,i,2}^S$ from $\text{Exp}_{4,i,1}^S$, then we can use this adversary to break DDH. Consequently, it holds that

$$\left| \Pr[\text{Exp}_{4,i,1}^S(\lambda) = 1] - \Pr[\text{Exp}_{4,i,2}^S(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

It is important to see that at this point, the only thing that the adversary has is $t = h_S \cdot Z$ where Z is uniformly random. His best option is to guess pwd^* , which is the only value that was sampled from a sub-exponential set. Per password guess $\overline{pwd}$, he obtains a potential $\overline{h_R} \leftarrow t / H_S(un^*, \overline{pwd}, n^*)$. Because the correct $h_R = H_R(un^*, n^*)^{sk_R} = Z$ is uniformly random, the adversary can only tell if a guess was correct by interacting with the honest ratelimiter in the validation protocol modeled by the validation oracle. We show that the adversary can only test one $\overline{h_R}$ per interaction with the honest ratelimiter. By sending $\alpha \leftarrow H_1(\overline{h_R})^r$ to the ratelimiter, the adversary learns whether $\overline{h_R} = Z$ because of the following equation:

$$\begin{aligned} y &\stackrel{?}{=} y' \\ H_2(Z, H_1(Z)^k) &\stackrel{?}{=} H_2(\overline{h_R}, H_1(\overline{h_R})^k). \end{aligned}$$

In the next and last hybrid experiment, we show that the adversary only learns this equality for exactly the $\overline{h_R}$ that he used in the computation of α . If he can somehow compute $H_2(H_1(\overline{h_R}), H_1(\overline{h_R})^k)$ for $\overline{h_R} \neq h_R$, he breaks the OM-gapDH assumption.

$\text{Exp}_{4,i,3}^S(\lambda)$. In this experiment, only the validation oracle involving un^* is changed. Upon receiving (α, n^*) from the adversary, the ratelimiter samples a random $a \leftarrow \mathbb{Z}_q^*$ and computes $\beta \leftarrow \text{Help}(x)^a$. This implicitly sets $k \leftarrow K \cdot a$ with K being the internal secret of the OM-gapDH assumption. The ratelimiter samples y uniformly at random. To ensure that this is indistinguishable from $\text{Exp}_{4,i,3}^S(\lambda)$, we have to program $H_2(h_R, H_1(h_R)^k)$ to be y . We recognize such a query by recording all a that we used to implicitly compute $k \leftarrow K \cdot a$ and checking if for the entered query B , it holds that $B^{a^{-1}} = H_1(h_R)^k$ for all recorded a . This check can be done with the DDH oracle of the OM-gapDH assumption. Because H_1 only returns challenge elements from Targ , every $B^{1/a}$ is a solution if the above equation holds. If the adversary can compute $H_2(\widetilde{h}_R, H_1(\widetilde{h}_R)^k)$ for $\widetilde{h}_R$ that he did not use in a previous validation oracle query, we get an additional solution, abort and break the OM-gapDH assumption. $\text{Exp}_{4,i,3}^S$ is only distinguishable from $\text{Exp}_{4,i,2}^S$ if the game aborts which happens only if we break OM-gapDH and, therefore, we get

$$\left| \Pr[\text{Exp}_{4,i,2}^S(\lambda) = 1] - \Pr[\text{Exp}_{4,i,3}^S(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

Because the validation oracle can only be queried Q_{limit} times for un^* until the ratelimiter refuses to answer (with Q_{limit} being the rate-limit implicitly used in the **ratelimit** function), the adversary can test at most Q_{limit} password guesses using the validation oracle and we get

$$\Pr[\text{Exp}_{4,i,3}^S(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda).$$

Case 2: Corruption of R.

$\text{Exp}_{4,i,0}^R(\lambda)$. This experiment is equivalent to $\text{Exp}_{4,i}$.

$\text{Exp}_{4,i,1}^R(\lambda)$. In this experiment, we do not sample a new server secret key sk_S during the key rotation. Instead, we forward every en-/decryption with sk_S to the corresponding oracles of the CCA-game. During the update of a ciphertext that, once decrypted with the old server key $\bar{sk}_S$, reveals n^* , we replace t with a uniformly random group element. During the validation protocol, we still use the original t instead of the uniformly random one encrypted in the enrollment record to ensure correctness. If there exists an adversary that distinguishes $\text{Exp}_{4,i,1}^R$ from $\text{Exp}_{4,i,0}^R$, then we can use this adversary to break the CCA-security of the encryption scheme. Consequently, it holds that

$$\left| \Pr[\text{Exp}_{4,i,0}^R(\lambda) = 1] - \Pr[\text{Exp}_{4,i,1}^R(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

Since t is uniformly random, the adversary learns nothing from T^* . In the validation oracle, all the ratelimiter learns from the server is a uniformly random value α and hence learns nothing from the protocol transcript. At the end of a validation oracle, the adversary also learns the output of the server, which is

only 1 if it entered the correct password. The only way left to win is to make the server output 1 for an incorrect password in the solve oracle. The server only outputs 1 if

$$y = y'$$

holds. The adversary knows h_R and has free choice over β and y . It also knows pwd' entered into the oracle and hence can compute $H_5(un^*, pwd', n^*)$. Because it does not know t , it does not know h'_R . The probability of guessing h'_R correctly so that

$$y = H_2(h'_R, \beta^{r^{-1}})$$

holds is negligible. Therefore, the only option left is to follow the protocol and enter the correct password into the solve oracle. He succeeds with probability

$$\Pr[\text{Exp}_{4,i,1}^R(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda).$$

Cases 1 & 2 combined. Because the limits of the **ratelimit** functions are synced during each key rotation, the adversary can guess the password Q_{limit} -times in total. Hence, we get

$$\Pr[\text{Exp}_{4,i}(\lambda) = 1] \leq \frac{Q_{\text{limit}}}{|\mathcal{PW}|} + \text{negl}(\lambda)$$

and therefore, we conclude the proof.

Compiler for Adaptive Security. We prove the adaptive security of **lrM-GUARD** by showing that if a PPT adversary $\mathcal{A}$ has non-negligible advantage $\epsilon_{\mathcal{A}}$ above the trivial guessing probability in the fully-adaptive unforgeability experiment, we can construct a PPT adversary $\mathcal{B}$ having non-negligible advantage above the trivial guessing probability against semi-adaptive unforgeability (the formal definition of semi-adaptive security can be found in the full version of this paper [3]). $\mathcal{B}$ runs $\mathcal{A}$ as a subroutine, and simulates the fully-adaptive oracles using the semi-adaptive ones. The two experiments are almost identical, except in how they handle key rotation and corruption. Therefore, $\mathcal{B}$ merely acts as a forwarder for all oracle calls except for the corruption-related ones and a subtle edge case described below.

To handle rotation and corruption queries, $\mathcal{B}$ predicts a corruption choice before each honest key rotation. It uses this prediction to call the semi-adaptive rotation oracle and obtain the secret key sk_i . If $\mathcal{A}$ corrupts the predicted party in the next epoch, $\mathcal{B}$ returns sk_i . If $\mathcal{A}$ corrupts the other party, $\mathcal{B}$ rewinds $\mathcal{A}$ to the most recent rotation call and repeats the process with a new guess.

This procedure is attempted up to λ times, until one attempt reaches the end without rewinding or the limit is reached. For a fixed epoch and a single attempt, the probability that the corruption guess was correct is at least $\frac{1}{2}$, and thus the probability of failing λ times in a row is $\leq (1 - \frac{1}{2})^\lambda \leq 2^{-\lambda}$. The upper bound to the number of rotations is Q , and because rotations are triggered by

$\mathcal{A}$ and $\mathcal{A}$ is PPT, Q has to be polynomial, and thus the probability of failure over the course of all epochs is still negligible. Because we have to rewind $Q \cdot \lambda$ times, this results in a security loss of $Q \cdot \lambda$.

The oracles $\mathcal{O}_{\text{enr1Ch}}$, $\mathcal{O}_{\text{valCh}}$, and $\mathcal{O}_{\text{Solve}}$ present subtle edge cases because they abort if the server is corrupted. If $\mathcal{A}$ queries them before corrupting the server within the same epoch, $\mathcal{B}$ cannot forward the calls to the semi-adaptive game, because in the semi-adaptive game, the server is corrupt during the whole epoch. However, $\mathcal{O}_{\text{Solve}}$ remains unaffected as the game terminates before any subsequent corruption occurs. Calls to $\mathcal{O}_{\text{valCh}}$ can be simulated by calling $\mathcal{O}_{\text{val}}(T, un^*, \epsilon)$, which is basically the same except that $\mathcal{B}$ uses an incorrect password. This is indistinguishable by $\mathcal{A}$ as it learns nothing from $\mathcal{O}_{\text{valCh}}$ anyway.

To answer the call to $\mathcal{O}_{\text{enr1Ch}}$, which only happens once, $\mathcal{B}$ guesses the preceding honest key rotation and whether the server will be corrupted in that epoch. If so, $\mathcal{B}$ calls $\mathcal{O}_{\text{enr1Ch}}$ after the key rotation, stores T^* , and invokes another key rotation, followed by a call to $\mathcal{O}_{\text{Update}}$ using T^* . Once $\mathcal{A}$ calls $\mathcal{O}_{\text{enr1Ch}}$, $\mathcal{B}$ returns the updated T^* . Normal calls to $\mathcal{O}_{\text{enr1Ch}}$ are answered by calling it twice for each key rotation’s update token. The security loss coming from guessing the epoch in the corruption in that epoch is $Q \cdot 2$.

The total security loss in comparison to the semi-adaptive security of `IrmGUARD` results in $Q \cdot 2 \cdot Q \cdot \lambda = 2\lambda Q^2$, which is polynomial. Finally, since $\mathcal{A}$ is efficient, and we rewind at most $(q \cdot \lambda)$ times, $\mathcal{B}$ is also efficient.

5.2 Prototype and Benchmark

We benchmark the performance of `IrmGUARD` against `HildeGuard` [5], the most efficient PHE scheme, and `Phoenix` [17], the previously most efficient PH scheme. We do not compare performance against `PW-Hero`, since `PW-Hero` is insecure also in the static corruption model.

Prototype Implementation. We implement our prototype in `Rust 1.86.0-nightly`. For elliptic-curve operations, we use the `curve25519-dalek` crate (version 4.1.3), `sha2` (version 0.10) for hashing, and `aes-gcm` (version 0.10) for symmetric encryption. All benchmarks were executed on a MacBook Pro (Apple M3 Pro, 12-core CPU, 36 GB RAM) running macOS Tahoe 26.0.1 using Criterion’s default configuration, i.e. 100 samples and 5 % warmup. We publish our prototype at <https://anonymous.4open.science/r/2025-IrmGuard-src-EE3B/> for the review process and intend to make it open-source upon publication.

Microbenchmarks. We evaluate the performance of each component in the enrollment and validation workflows separately for both the *server* and the *ratelimiter*. Criterion automatically repeats each benchmark for 100 iterations and reports both the median latency and the 95th-percentile (p95) latency. The p95 value indicates that 95% of all runs completed in less than this time, providing an upper bound on typical latency. We depict the results in Table 1. The most computationally intensive operations are `ratelimiter_validate` (92.59 μs , dominated by elliptic-curve exponentiations) and `ratelimiter_enroll` (64.3 μs), dominated

Table 1. Microbenchmarks of `lrmGUARD` on a MacBook Pro (M3 Pro, 36 GB RAM, macOS Tahoe 26.0.1). Each row reports the median and 95th-percentile (p95) execution time over 100 Criterion iterations (single thread).

Operation	Median [μ s]	p95 [μ s]
Server enroll_init	0.74	0.74
Ratelimiter enroll	64.30	64.36
Server enroll_finish	47.05	47.34
Server validate_init	39.82	39.86
Ratelimiter validate	92.59	93.68
Server validate_finish	39.17	39.28
Schnorr proof generation	35.63	36.26
Schnorr proof verification	35.07	35.11

Table 2. Theoretical throughput from median latencies. “Net adj.” multiplies by 0.65 to account for network overhead. Measurements on a MacBook Pro (M3 Pro, 36 GB RAM).

Component	Latency [μ s]	Throughput [ops/s]	Net adj. ($\times 0.65$)
Server enroll (init+finish)	47.79	20,920	13,598
Server validate (init+finish)	78.99	12,658	8,227
Ratelimiter enroll	64.30	15,551	10,108
Ratelimiter validate	92.59	10,799	7,019

by proof generation. All operations are below 100μ s, and the p95 percentiles are close to the median ($p < 0.05$) for all operations except ratelimiter validation. This deviation is expected, as ratelimiter validation involves multiple hash-to-curve operations and scalar multiplications, which introduce slightly higher variance.

Estimated Throughput. To assess practical scalability, we translate the measured latencies into an approximate end-to-end throughput. Assuming sequential execution, we estimate throughput as the inverse of the median latency. Combining the `init` and `finish` phases, the *server* achieves an estimated throughput of approximately 20,900 operations per second for enrollment and 12,700 operations per second for validation. The *ratelimiter* achieves around 15,600 operations per second for enrollment and 10,800 operations per second for validation.

To account for network overhead and serialization costs that occur in practice, we conservatively scale these values by 0.65, resulting in an effective throughput of roughly 7,000–13,600 operations per second. We depict the results in Table 2.

Comparison to existing schemes. We compare `lrmGUARD` with Phoenix [17] and HildeGuard [5]. To this extent, we reimplemented Phoenix [17] and cloned the public HildeGuard repository [4] and re-ran the same microbenchmarks for these

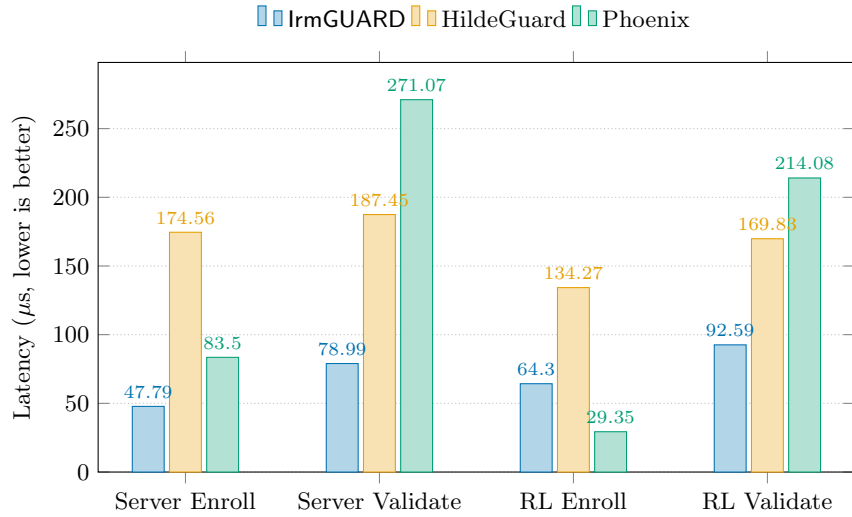


Fig. 11. Combined median latencies (Criterion) for matching stages (smaller is better). Server bars sum `init`+`finish`; Ratelimiter bars are measured directly.

other protocols on the *same* machine and software stack as IrmGUARD (MacBook Pro, M3 Pro, 36 GB RAM, macOS Tahoe 26.0.1). We compare matching algorithmic stages: for each scheme, we combine the server’s `init` and `finish` times (enrollment and validation), and we report the ratelimiter’s enrollment and validation as measured. We depict the results in Figure 11.

Our results show that IrmGUARD achieves the lowest latency in the validation phase, which is the dominant operation in practice as validations occur significantly more frequently than enrollments. In particular, IrmGUARD clearly outperforms both Phoenix and HildeGuard in server-side validation and yields lower ratelimiter validation costs than both schemes. This result is particularly surprising in the case of Phoenix because it was proven secure in a weaker security model, which naturally allows for more efficient instantiations.

For enrollment, IrmGUARD is twice as efficient as Phoenix on the server side while significantly outperforming HildeGuard; however, Phoenix remains faster for ratelimiter enrollment. These results align with the design goals: IrmGUARD and Phoenix are lightweight password-hardening (PH) schemes optimized for frequent online authentication, whereas HildeGuard is a full password-hardened encryption (PHE) construction with additional cryptographic guarantees, resulting in higher computational overhead.

Ethical Considerations

Our work studies the security of password-hardening protocols under stronger corruption patterns than those considered in prior analyses. In particular, we

show that implementations of Phoenix become insecure when an adversary first corrupts the ratelimiter and later corrupts the server after an honest key rotation. This attack does not contradict the formal proof of Phoenix, which is stated for a weaker adversary model. Rather, our result shows that this adversary model is too weak to capture practically relevant attacks. A second attack demonstrates how to bypass PW-Hero’s rate limiting. This attack is more severe, as it shows that the security of PW-Hero is broken even under the static corruption model.

The stakeholders affected by this work are protocol designers, implementers, and operators of authentication systems, as well as users whose passwords may be exposed to online guessing attacks if such systems are deployed under inadequate assumptions. The main benefit of this work is to clarify the limits of the security guarantees provided by Phoenix and to help prevent deployments based on overly optimistic threat models. This also motivates the stronger design goals of our scheme.

We are not aware of any real-world deployment of Phoenix or PW-Hero. For this reason, we believe that presenting the attacks at the level of reference implementations is ethically justified and unlikely to cause concrete harm to users. Moreover, we do not provide a turnkey exploit against a live deployment. Instead, we show only what an attacker who has already obtained access to the relevant system components can do after such a compromise. In other words, our contribution demonstrates the security consequences of an already successful corruption, rather than introducing a new intrusion vector.

We did not conduct experiments on third-party systems, did not access any real user accounts or credentials, and did not interact with live services. All experiments were carried out locally in a controlled test environment against our own implementation. Thus, the work did not expose uninvolved users, operators, or infrastructures to additional risk.

Overall, we believe that the defensive value of documenting these limitations outweighs the risks of publication. Making the attack explicit helps protocol designers and evaluators avoid relying on insufficient threat models and supports the development of schemes with stronger corruption resilience.

References

1. Behzad Abdolmaleki, Ruben Baecker, Paul Gerhart, Mike Graf, Mojtaba Khalili, Daniel Rausch, and Dominique Schröder. Universally composable password-hardened encryption. In *ASIACRYPT 2025*, LNCS. Springer, 2025.
2. Shashank Agrawal, Peihan Miao, Payman Mohassel, and Pratyay Mukherjee. PASTA: PASsword-based threshold authentication. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 2042–2059. ACM Press, October 2018.
3. Ruben Baecker, Paul Gerhart, and Dominique Schröder. Breaking and Fixing Password-Hardening. [https://www.ruben-baecker.de/paper/breaking_and_fixing_ph.pdf; accessed 21-April-2026].
4. Ruben Baecker, Paul Gerhart, and Dominique Schröder. Hildegard: A semi-adaptively secure password-hardened encryption scheme. GitHub, 2025. GitHub repository.

5. Ruben Baecker, Paul Gerhart, and Dominique Schröder. Password-hardened encryption revisited. In *ASIACRYPT 2025*, LNCS. Springer, 2025.
6. Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011*, pages 433–444. ACM Press, October 2011.
7. Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Heidelberg, May 2000.
8. Julian Brost, Christoph Egger, Russell W. F. Lai, Fritz Schmid, Dominique Schröder, and Markus Zoppelt. Threshold password-hardened encryption services. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 409–424. ACM Press, November 2020.
9. Jan Camenisch, Anja Lehmann, and Gregory Neven. Optimal distributed password verification. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 182–194. ACM Press, October 2015.
10. David Chaum and Torben P. Pedersen. Wallet databases with observers. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 89–105. Springer, Heidelberg, August 1993.
11. Alfredo De Santis, Silvio Micali, and Giuseppe Persiano. Non-interactive zero-knowledge proof systems. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 52–72. Springer, Heidelberg, August 1988.
12. Constantinos Diomedous and Elias Athanasopoulos. Practical password hardening based on tls. In Roberto Perdisci, Clémentine Maurice, Giorgio Giacinto, and Magnus Almgren, editors, *Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 441–460, Cham, 2019. Springer International Publishing.
13. Adam Everspaugh, Rahul Chatterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. The pythia PRF service. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 547–562. USENIX Association, August 2015.
14. Stanislaw Jarecki, Hugo Krawczyk, Maliheh Shirvanian, and Nitesh Saxena. Two-factor authentication with end-to-end password security. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 431–461. Springer, Heidelberg, March 2018.
15. Chunfu Jia, Shaoqiang Wu, and Ding Wang. Reliable Password Hardening Service with Opt-Out. In *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*, pages 250–261, Los Alamitos, CA, USA, September 2022. IEEE Computer Society.
16. Russell W. F. Lai, Christoph Egger, Manuel Reinert, Sherman S. M. Chow, Matteo Maffei, and Dominique Schröder. Simple password-hardened encryption services. In William Enck and Adrienne Porter Felt, editors, *USENIX Security 2018*, pages 1405–1421. USENIX Association, August 2018.
17. Russell W. F. Lai, Christoph Egger, Dominique Schröder, and Sherman S. M. Chow. Phoenix: Rebirth of a cryptographic password-hardening service. In Engin Kirda and Thomas Ristenpart, editors, *USENIX Security 2017*, pages 899–916. USENIX Association, August 2017.
18. Jonas Schneider, Nils Fleischhacker, Dominique Schröder, and Michael Backes. Efficient cryptographic password hardening services from partially oblivious commitments. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1192–1203. ACM Press, October 2016.

A Formal Oracle Definition of Our Semi-Adaptive Confidentiality Definition

$\mathcal{O}_{\text{enr1}}(un, pwd)$ <pre> 1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 2 : $\text{else } S(sk_S, un, pwd)$ 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 4 : $\text{else } R(sk_R, un)$ 5 : $(T, \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{enr1}}$ 6 : return T </pre>	$\mathcal{O}_{\text{val}}(T, un, pwd)$ <pre> 1 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 2 : $\text{else } S(sk_S, T, un, pwd)$ 3 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 4 : $\text{else } R(sk_R, un)$ 5 : $(b, \epsilon) \leftarrow \langle S^*, R^* \rangle_{\text{val}}$ 6 : return b </pre>
$\mathcal{O}_{\text{enr1Ch}}()$ <pre> 1 : ensure $\text{Corr} \neq S \wedge T_j^* = \epsilon \forall j$ 2 : $T_i^* \leftarrow \mathcal{O}_{\text{enr1}}(un^*, pwd^*)$ 3 : return T_i^* </pre>	$\mathcal{O}_{\text{valCh}}(T)$ <pre> 1 : ensure $\text{Corr} \neq S$ 2 : $\epsilon \leftarrow \mathcal{O}_{\text{val}}(T, un^*, pwd^*)$ 3 : return ϵ </pre>
$\mathcal{O}_{\text{HonRot}}(\text{Corr}')$ <pre> 1 : ensure $\text{Corr}' \in \{S, R, \perp\}$ 2 : $S^* := S(sk_S)$ 3 : $R^* := R(sk_R)$ 4 : $((k_S, \tau'), k_R) \leftarrow \langle S^*, R^* \rangle_{\text{rot}}$ 5 : $(pk_R, sk_R) \leftarrow k_R, i \leftarrow i + 1$ 6 : $(pk_S, sk_S) \leftarrow k_S, \tau_i \leftarrow \tau'$ 7 : $\text{Corr} \leftarrow \text{Corr}'$ 8 : if $\text{Corr} = S$ then return sk_S 9 : if $\text{Corr} = R$ then return sk_R 10 : return $\perp$ </pre>	$\mathcal{O}_{\text{DishonRot}}()$ <pre> 1 : ensure $\text{Corr} \in \{S, R\}$ 2 : $S^* := \text{if } \text{Corr} = S \text{ then } \mathcal{A}$ 3 : $\text{else } S(sk_S)$ 4 : $R^* := \text{if } \text{Corr} = R \text{ then } \mathcal{A}$ 5 : $\text{else } R(sk_R)$ 6 : $((k_S, \tau'), k_R) \leftarrow \langle S^*, R^* \rangle_{\text{rot}}$ 7 : $(pk_R, sk_R) \leftarrow k_R, i \leftarrow i + 1$ 8 : $(pk_S, sk_S) \leftarrow k_S, \tau_i \leftarrow \tau'$ 9 : return ϵ </pre>
$\mathcal{O}_{\text{Solve}}(pwd)$ <pre> 1 : ensure $\text{Corr} \neq S \wedge T_i^* \neq \epsilon$ 2 : if $\mathcal{O}_{\text{val}}(un^*, pwd, T_i^*)$ then 3 : end game with 1 4 : else end game with 0 </pre>	$\mathcal{O}_{\text{Update}}(j, T, un)$ <pre> 1 : ensure $\tau_j \neq \epsilon$ 2 : $T' \leftarrow \text{Update}(sk_{Sj}, \tau_j, T, un)$ 3 : if $T = T_{j-1}^* \wedge un = un^*$ then 4 : $T_j^* \leftarrow T'$ 5 : return T' </pre>

Fig. 12. Oracles for the semi-adaptive PH confidentiality definition.